



KeyCraft Retro

Подробный разбор кода: из чего состоит проект, где и как хранятся данные, как работают React-хуки, состояние, контексты, навигация и каждый компонент.

Как читать этот документ

- Разделы 1–3 — общая картина: стек, запуск и структура папок.
- Разделы 4–6 — основы React на примерах из этого проекта: компоненты, props, state и все используемые хуки.
- Разделы 7–9 — **где хранится каждое значение**: состояние компонентов, контексты, localStorage, sessionStorage, адресная строка.
- Разделы 10–13 — разбор каждого файла и сценарии «что происходит, когда пользователь нажимает...».
- Разделы 14–16 — стили, инструменты, слабые места и словарь терминов.

Весь код в рамках — это настоящие фрагменты из репозитория (подпись над рамкой показывает файл и номера строк).

Содержание

- 1 Что это за проект и на чём он написан
- 2 Как приложение запускается: от index.html до экрана
- 3 Структура папок и файлов
- 4 Основы React на примерах проекта: компонент, JSX, props, state
- 5 Хуки React: useState, useEffect, useLayoutEffect, useRef, useMemo, useCallback, useContext
- 6 Собственные хуки проекта: usePersistentState и useTimedValue
- 7 Где хранится состояние: пять уровней хранения
- 8 Глобальное состояние: CartContext и FavoritesContext
- 9 Данные магазина: products.js
- 10 Навигация без библиотек: hash-роутинг и память прокрутки
- 11 Дерево компонентов
- 12 Разбор компонентов по файлам
- 13 Сценарии: что происходит при действиях пользователя
- 14 Стили: Tailwind CSS и index.css
- 15 Инструменты: Vite, npm-скрипты, линтер
- 16 Слабые места и что можно улучшить
- 17 Словарик терминов

1 Что это за проект

KeyCraft Retro — интернет-магазин кастомных механических клавиатур. Это **одностраничное приложение (SPA)**: браузер один раз загружает HTML и JavaScript, а дальше все «страницы» (главная, каталог, товар, избранное) рисуются JavaScript-кодом без перезагрузки.

Что умеет сайт

- Главная с интерактивной 3D-моделью клавиатуры, которую можно разобрать по слоям и покрутить мышкой.
- Карусель спецпредложений, быстрые ссылки на категории.
- Каталог из 24 товаров: поиск, фильтр по категории и цене, сортировка, пагинация, вид «сетка/список».
- Страница товара: выбор варианта (например, свитчей), количество, вкладки «Характеристики / Комплектация / Отзывы», похожие товары.
- Избранное, корзина с промокодами и прогрессом до бесплатной доставки, оформление заказа с чеком и конфетти.
- Чат-бот «консультант» с готовыми сценариями.

Важно: у проекта нет сервера (бэкенда)

Товары лежат прямо в коде (`src/data/products.js`), корзина и избранное сохраняются **в браузере пользователя** (localStorage), заказы никуда не отправляются — тоже пишутся в localStorage. Чат-бот — не нейросеть, а набор правил «ключевое слово → ответ». Для учебного/демо-проекта это нормально, но для настоящего магазина понадобится сервер (см. раздел 16).

Технологии (стек)

Ни Redux, ни React Router, ни UI-библиотек компонентов нет — всё сделано встроенными средствами React. Поэтому проект хорошо подходит, чтобы разобраться в «чистом» React.

Технология	Версия	Зачем нужна
React + react-dom	19.2	Библиотека для построения интерфейса из компонентов. Сама следит, что поменялось, и перерисовывает только нужное.
Vite	8.3	Сборщик и дев-сервер. Превращает JSX в обычный JavaScript, мгновенно перезагружает страницу при правках, собирает папку <code>dist/</code> для публикации.
@vitejs/plugin-react	6.1	Плагин Vite: поддержка JSX и «горячей» перезагрузки компонентов (Fast Refresh).
Tailwind CSS	3.4	CSS-фреймворк «утилитарных классов»: стили пишутся прямо в разметке (<code>px-4 rounded-lg bg-white</code>).
PostCSS + autoprefixer	8.5 / 10.6	Обрабатывают CSS: запускают Tailwind и добавляют вендорные префиксы для старых браузеров.
lucide-react	1.46	Набор SVG-иконок в виде React-компонентов: <code><ShoppingBag /></code> , <code><Heart /></code> и т.д.

canvas-confetti	1.9	Анимация конфетти после оформления заказа.
oxlint	1.81	Линтер — проверяет код на ошибки и нарушения правил хуков.

2 Как приложение запускается

Цепочка запуска короткая — четыре файла. Разберём, что делает каждый.



Рис. 1. От HTML-файла до главного компонента магазина

Шаг 1. index.html — пустая «рамка»

Браузер получает почти пустую страницу: заголовок, мета-теги, подключение шрифтов Google Fonts и один пустой контейнер `<div id="root">`. Последняя строка подключает JavaScript-модуль — с него всё начинается.

index.html · строки 14–17

```
<body class="bg-cream-100 text-vintage-dark antialiased selection:bg-vintage-accent selection:text-white min-h-screen">
  <div id="root"></div>
  <script type="module" src="/src/main.jsx"></script>
</body>
```

Шаг 2. src/main.jsx — точка входа

src/main.jsx · весь файл, 10 стр.

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.jsx'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

- `import './index.css'` — подключает глобальные стили (Tailwind + свои классы). Vite сам вставит их на страницу.
- `createRoot(document.getElementById('root'))` — React «захватывает» пустой div и дальше полностью управляет его содержимым.
- `.render(<App />)` — рисует корневой компонент.
- `<StrictMode>` — режим проверок **только при разработке**: React нарочно вызывает компоненты и эффекты дважды, чтобы выявить ошибки (например, изменение состояния «на месте»). В собранной версии сайта он ничего не делает.

Шаг 3. src/App.jsx — провайдеры и «оболочка» магазина

src/App.jsx · строки 246–254

```
export default function App() {
  return (
    <CartProvider>
      <FavoritesProvider>
        <MainShop />
      </FavoritesProvider>
    </CartProvider>
  );
}
```

`App` оборачивает магазин в два **провайдера контекста**. Провайдер — это компонент, который «раздаёт» данные всем вложенным компонентам на любой глубине. Благодаря этому корзина и избранное доступны в любом месте (подробно — раздел 8). Сам магазин — компонент `MainShop` в том же файле: он хранит текущую страницу и решает, что показать.

Шаг 4. Сборка для публикации

Команда `npm run build` запускает Vite: он переводит JSX в обычный JS, собирает все модули в один файл `dist/assets/index-*.js` (~375 КБ, ~113 КБ в gzip), а CSS — в `dist/assets/index-*.css`. Tailwind при этом оставляет только те классы, которые реально встречаются в коде. Папку `dist/` можно выложить на любой статический хостинг (в README описан деплой на Vercel).

3 Структура папок и файлов

keycraft-retro/	
├─ index.html	HTML-«рамка»: <div id="root"> + подключение main.jsx
├─ package.json	зависимости и npm-скрипты (dev / build / lint / preview)
├─ vite.config.js	настройка сборщика (подключён плагин React)
├─ tailwind.config.js	тема Tailwind: цвета cream/vintage, шрифты, тени, анимации
├─ postcss.config.js	запуск Tailwind и autoprefixer
├─ .oxlinttrc.json	правила линтера (rules-of-hooks — ошибка)
├─ public/images/	картинки товаров и промо-слайдов (SVG), отдаются «как есть»
├─ src/	
├─├─ main.jsx	точка входа: createRoot + StrictMode
├─├─ App.jsx	провайдеры, навигация, выбор страницы, главная
├─├─ index.css	Tailwind + базовые стили + классы btn-retro, card-retro...
├─├─ data/	
├─├─├─ products.js	ВСЕ данные: категории, 24 товара, промо-слайды, промокоды
├─├─ context/	глобальное состояние (доступно везде)
├─├─├─ CartContext.jsx	корзина, промокод, итоги, заказ, тосты, открыта ли корзина
├─├─├─ FavoritesContext.jsx	список избранного
├─├─ hooks/	собственные хуки
├─├─├─ usePersistentState.js	useState + автосохранение в localStorage/sessionStorage
├─├─├─ useTimedValue.js	значение, которое само сбрасывается через N мс
├─├─ utils/	
├─├─├─ format.js	rub(18900) → "18 900 ₽"
├─├─ components/	компоненты интерфейса
├─├─├─ Navbar.jsx	шапка + нижняя мобильная панель
├─├─├─ HeroExplodedParallax.jsx	3D-герой на главной (управление, анимация)
├─├─├─ HeroKeyboard3D.jsx	6 SVG-слоёв клавиатуры
├─├─├─ PromoCarousel.jsx	карусель спецпредложений
├─├─├─ Breadcrumbs.jsx	«хлебные крошки» в каталоге
├─├─├─ Catalog/	
├─├─├─├─ Catalog.jsx	каталог: фильтрация, сортировка, пагинация
├─├─├─├─ FilterBar.jsx	панель фильтров + переключатель «сетка/список»
├─├─├─├─ ProductCard.jsx	карточка товара + список карточек
├─├─├─├─ Pagination.jsx	переключатель страниц
├─├─ ProductDetailPage.jsx	страница товара
├─├─ FavoritesPage.jsx	страница избранного
├─├─ CartDrawer.jsx	выезжающая панель корзины
├─├─ CheckoutModal.jsx	оформление заказа и чек
├─├─ ClackBotModal.jsx	чат-бот
├─├─ Toast.jsx	всплывающее уведомление
├─├─ Footer.jsx	подвал

Размер файлов

Файл	Строк	Что внутри
src/data/products.js	557	Категории, 24 товара, промо-слайды, промокоды
src/components/HeroExplodedParallax.jsx	469	3D-герой: данные слоёв, управление, анимация наклона
src/components/HeroKeyboard3D.jsx	458	Шесть SVG-слоёв клавиатуры, сгенерированных из таблиц
src/components/ProductDetailPage.jsx	397	Страница товара
src/App.jsx	254	Провайдеры, навигация, главная страница
src/components/ClackBotModal.jsx	246	Чат-бот: правила и окно чата

<code>src/components/CheckoutModal.jsx</code>	241	Форма заказа, валидация, чек
<code>src/components/CartDrawer.jsx</code>	205	Панель корзины
<code>src/components/Catalog/ProductCard.jsx</code>	184	Карточка товара + ProductList
<code>src/components/Navbar.jsx</code>	179	Шапка и нижняя мобильная панель
<code>src/components/Catalog/Catalog.jsx</code>	162	Каталог: фильтр, сортировка, пагинация
<code>src/components/Catalog/FilterBar.jsx</code>	135	Панель фильтров + ViewModeToggle
<code>src/context/CartContext.jsx</code>	128	Глобальное состояние корзины
<code>src/components/PromoCarousel.jsx</code>	121	Карусель спецпредложений
<code>src/components/FavoritesPage.jsx</code>	82	Страница избранного
<code>src/components/Footer.jsx</code>	79	Подвал
<code>src/index.css</code>	74	Tailwind + базовые стили + свои классы
<code>src/components/Catalog/Pagination.jsx</code>	59	Переключатель страниц каталога
<code>src/components/Breadcrumbs.jsx</code>	42	Хлебные крошки
<code>src/components/Toast.jsx</code>	35	Всплывающее уведомление
<code>src/context/FavoritesContext.jsx</code>	27	Глобальное состояние избранного
<code>src/hooks/usePersistentState.js</code>	23	Хук: useState + localStorage/sessionStorage
<code>src/hooks/useTimedValue.js</code>	17	Хук: значение с автосбросом по таймеру
<code>src/main.jsx</code>	10	Точка входа
<code>src/utils/format.js</code>	1	Форматирование цены

Принцип организации

data — «что продаём», **context** — «общее состояние», **hooks** — «переиспользуемая логика», **utils** — «чистые функции», **components** — «как это выглядит». Компоненты не хранят товары у себя — они импортируют их из `data/products.js`.

4 Основы React на примерах проекта

4.1 Компонент

Компонент — это обычная функция JavaScript, которая возвращает разметку. Имя всегда с большой буквы. Вот самый маленький компонент проекта — бейдж-счётчик в мобильной панели:

src/components/Navbar.jsx · строки 173–179

```
function MobileBadge({ bg, children }) {  
  return (  
    <span className={`absolute -top-0.5 right-1.5 ${bg} text-white text-[9px] font-bold px-1 rounded-full min-w-[14px] text-center`} >  
      {children}  
    </span>  
  );  
}
```

Его используют как HTML-тег: `<MobileBadge bg="bg-red-500">{favoritesCount}</MobileBadge>`. Всё, что передано атрибутами, приходит в функцию одним объектом — **props** (здесь `bg` и `children` — содержимое между тегами).

4.2 JSX

Разметка внутри JS называется **JSX**. Это не HTML, а удобная запись вызовов React. Отличия, которые встречаются в проекте:

В JSX	Почему так
<code>className="..."</code>	Слово <code>class</code> занято в JavaScript.
<code>{product.title}</code>	Фигурные скобки — «вставить значение JS».
<code>{cond && <div/>}</code>	Условный показ: если <code>cond</code> ложно — ничего не рисуется. Пример: <code>{product.badge && <div>...</div>}</code> .
<code>{a ? <X/> : <Y/>}</code>	Выбор из двух вариантов (кнопка «В корзину» или счётчик).
<code>{list.map(item => <Li key={item.id}/>)}</code>	Список. key обязателен: по нему React понимает, какой элемент какой, при перестановках.
<code>onClick={() => open(id)}</code>	Обработчик события — функция, а не строка.
<code>style={{ transform: '...' }}</code>	Стили объектом (двойные скобки: внешние — JS, внутренние — объект).
<code><>...</></code>	Фрагмент — группа элементов без лишнего div.

4.3 Props — данные «сверху вниз»

Родитель передаёт данные и функции дочерним компонентам. Например, `App` передаёт каталогу текущую категорию и функцию открытия товара:

```

<Catalog
  selectedCategory={selectedCategory}
  onSelectCategory={setSelectedCategory}
  searchQuery={searchQuery}
  onSearchChange={setSearchQuery}
  onOpenProduct={openProduct}
/>

```

Каталог сам **не умеет** переключать страницы — он просто вызывает полученную функцию `onOpenProduct(id)`, а `App` решает, что делать. Это главный принцип React: **данные идут вниз через props, события — вверх через функции-колбэки**. Имена колбэков по соглашению начинаются с `он...`.

4.4 State — память компонента

Состояние (state) — данные, которые меняются во время работы и от которых зависит картинка. Например, какая вкладка открыта на странице товара. Обычную переменную использовать нельзя: при каждой перерисовке функция компонента вызывается заново, и переменная обнулится. Поэтому есть хук `useState`:

src/components/ProductDetailPage.jsx · строки 54–54

```
const [activeTab, setActiveTab] = useState('specs');
```

- `activeTab` — текущее значение (сначала `'specs'`);
- `setActiveTab('reviews')` — меняет значение и **заставляет React перерисовать** компонент с новым значением.

4.5 Как работает перерисовка (рендер)

1. Пользователь нажал кнопку → вызвался `setSomething(новое)`.
2. React запоминает новое значение и планирует перерисовку этого компонента и всех его детей.
3. Функция компонента вызывается заново, возвращает новый JSX.
4. React сравнивает новый JSX со старым и меняет в настоящей странице (DOM) только то, что отличается.

Правило неизменяемости

Состояние нельзя менять «на месте» (`items.push(x)`, `item.quantity += 1`) — React не заметит изменения. Нужно создавать **новый** массив/объект. В проекте это везде делается через `...` (spread), `map` и `filter`:

```

setCartItems(prev => [...prev, newItem]);           // добавить
setCartItems(prev => prev.filter(i => i.id !== id));   // удалить
setCartItems(prev => prev.map(i => i.id === id ? { ...i, quantity: i.quantity + 1 } : i)); //
изменить

```

Форма `setX(prev => ...)` («функциональное обновление») берёт самое свежее значение — так безопаснее, если обновлений несколько подряд.

4.6 Подъём состояния и «контролируемые» компоненты

Если состояние нужно двум компонентам, его хранят у их общего родителя. Пример: строка поиска нужна и шапке (поле ввода), и каталогу (фильтрация). Поэтому `searchQuery` живёт в `MainShop`, а оба получают его через props.

`FilterBar`, `Pagination`, `Breadcrumbs`, `Footer` вообще не имеют своего состояния — они только показывают то, что им передали, и сообщают о нажатиях. Такие компоненты называют **контролируемыми** («глупыми»): их легко понимать и переиспользовать.

Поле ввода тоже бывает контролируемым: `value` берётся из state, а `onChange` обновляет state:

`src/components/CartDrawer.jsx` · строки 161–167

```
<input
  type="text"
  value={promoInput}
  onChange={(e) => setPromoInput(e.target.value)}
  placeholder="Промокод (VINTAGE10)"
  className="w-full bg-stone-50 border border-stone-300 rounded-lg py-2 pl-7 pr-2.5 text-xs font-mono
  focus:outline-none uppercase"
/>
```

5 Хуки React

Хук — функция, имя которой начинается с `use`, дающая компоненту «суперспособность»: память, побочные эффекты, доступ к контексту. В проекте используются 7 встроенных хуков и 2 собственных.

Два правила хуков (линтер проверяет их как ошибку)

1. Вызывать хуки только **на верхнем уровне** компонента — не внутри `if`, циклов и вложенных функций.
2. Вызывать только из компонентов или других хуков.

React различает хуки по **порядку вызова**. Поэтому в проекте все хуки стоят в начале функции, а «ранний выход» — после них:

`src/components/CartDrawer.jsx` · строки 14–29

```
const [promoInput, setPromoInput] = useState('');
const close = () => setIsCartOpen(false);

// Esc закрывает корзину, фон не скроллится
useEffect(() => {
  if (!isCartOpen) return;
  const onKey = (e) => e.key === 'Escape' && setIsCartOpen(false);
  window.addEventListener('keydown', onKey);
  document.body.style.overflow = 'hidden';
  return () => {
    window.removeEventListener('keydown', onKey);
    document.body.style.overflow = '';
  };
}, [isCartOpen, setIsCartOpen]);

if (!isCartOpen) return null;
```

Где какой хук используется

Хук	Для чего	Где в проекте
<code>useState</code>	Память компонента; изменение вызывает перерисовку	Почти везде: страница, вкладка, количество, форма заказа, слайд карусели, сообщения чат-бота...
<code>useEffect</code>	Действие <i>после</i> отрисовки: подписки, таймеры, localStorage	App (hashchange), Hero (мышь + анимация), Carousel (автопрокрутка), CartDrawer (Esc + блок скролла), ClackBot (прокрутка чата), Catalog (сброс страницы), usePersistentState, useTimedValue
<code>useLayoutEffect</code>	То же, но <i>до</i> того, как пользователь увидит кадр	App — восстановление прокрутки без «мигания»
<code>useRef</code>	Память без перерисовки; ссылка на DOM-элемент	App (позиции прокрутки), Hero (DOM-элементы + координаты мыши), ClackBot (конец чата), Catalog (прошлые фильтры), useTimedValue (id таймера)
<code>useMemo</code>	Запомнить результат тяжёлого вычисления	Catalog — отфильтрованный и отсортированный список
<code>useCallback</code>	Запомнить саму функцию между рендерами	useTimedValue — функция <code>show</code>

<code>createContext</code> / <code>useContext</code>	Общие данные без передачи через props	<code>CartContext</code> , <code>FavoritesContext</code> → хуки <code>useCart()</code> , <code>useFavorites()</code>
--	---------------------------------------	--

5.1 useState

Уже разобран в 4.4. Пара приёмов из проекта:

Состояние-объект (форма заказа)

`src/components/CheckoutModal.jsx` · строки 22–29

```
const [formData, setFormData] = useState({
  fullName: '',
  phone: '',
  email: '',
  city: 'Москва',
  address: '',
  deliveryMethod: 'cdek',
});
```

Вместо шести `useState` — один объект. При изменении поля копируем объект и меняем одно свойство: `setFormData({ ...formData, [name]: e.target.value })`. Квадратные скобки `[name]` — «вычисляемое имя свойства».

Ленивая инициализация

Если начальное значение дорого вычислять (например, читать `localStorage`), в `useState` передают **функцию** — React вызовет её только один раз при первом рендере. Так сделано в `usePersistentState` (раздел 6).

5.2 useEffect — «сделай после отрисовки»

Компонент должен только вычислять разметку. Всё, что «выходит наружу» (подписаться на событие окна, запустить таймер, записать в `localStorage`), делается в эффекте:

```
useEffect(() => {
  // 1. код эффекта — выполняется ПОСЛЕ того, как React обновил страницу
  return () => {
    // 2. очистка — перед следующим запуском эффекта и при удалении компонента
  };
}, [зависимости]); // 3. эффект перезапускается, только когда они изменились
```

Массив зависимостей	Когда запускается
<code>[]</code> — пустой	Один раз после первого появления компонента («монтаж»)
<code>[a, b]</code>	После первого рендера и каждый раз, когда изменилось <code>a</code> или <code>b</code>
не указан	После каждого рендера (в проекте не используется)

Пример 1: автопрокрутка карусели — таймер с очисткой

src/components/PromoCarousel.jsx · строки 13–19

```
useEffect(() => {
  if (isPaused) return;
  const interval = setInterval(() => {
    setCurrentIndex((prev) => (prev + 1) % SLIDES);
  }, 5000);
  return () => clearInterval(interval);
}, [isPaused]);
```

Когда мышь над слайдом, `isPaused` становится `true` → эффект перезапускается: сначала **очистка** останавливает старый интервал, потом новый запуск сразу выходит (`return`). Без очистки таймеры накапливались бы и слайды переключались всё быстрее.

Пример 2: корзина — подписка на клавишу Esc и блокировка прокрутки фона

src/components/CartDrawer.jsx · строки 17–27

```
// Esc закрывает корзину, фон не скроллится
useEffect(() => {
  if (!isCartOpen) return;
  const onKey = (e) => e.key === 'Escape' && setIsCartOpen(false);
  window.addEventListener('keydown', onKey);
  document.body.style.overflow = 'hidden';
  return () => {
    window.removeEventListener('keydown', onKey);
    document.body.style.overflow = '';
  };
}, [isCartOpen, setIsCartOpen]);
```

Пример 3: синхронизация с адресной строкой (один раз)

src/App.jsx · строки 54–65

```
// Синхронизация с адресной строкой (в т.ч. кнопки «Назад/Вперёд» браузера)
useEffect(() => {
  const syncFromHash = () => {
    const hash = window.location.hash.slice(1);
    if (hash.startsWith('product/')) go('product', hash.slice('product/'.length), { restore: true });
    else go(PAGES.includes(hash) ? hash : 'home', null, { restore: true });
  };
  syncFromHash();
  window.addEventListener('hashchange', syncFromHash);
  return () => window.removeEventListener('hashchange', syncFromHash);
  // eslint-disable-next-line react-hooks/exhaustive-deps
}, []);
```

5.3 useLayoutEffect

Работает как `useEffect`, но выполняется **до** того, как браузер покажет новый кадр. Используется, когда эффект меняет то, что видно. В `App` так восстанавливается прокрутка: если делать это в обычном `useEffect`, пользователь на долю секунды увидел бы страницу в начале, а потом «прыжок».

src/App.jsx · строки 48–52

```
useLayoutEffect(() => {
  if (pendingScroll.current === null) return;
  window.scrollTo({ top: pendingScroll.current, behavior: 'instant' });
  pendingScroll.current = null;
}, [currentPage, activeProductId]);
```

5.4 useRef — память без перерисовки

`const ref = useRef(начальное)` возвращает объект `{ current: ... }`, который живёт всё время жизни компонента. Менять `ref.current` можно сколько угодно — React **не перерисовывает** компонент. Два применения:

а) Ссылка на DOM-элемент

`src/components/ClackBotModal.jsx` · строки 79–83

```
useEffect(() => {
  if (!isOpen) return;
  const t = setTimeout(() => messagesEndRef.current?.scrollIntoView({ behavior: 'smooth' }), 100);
  return () => clearTimeout(t);
}, [isOpen, messages]);
```

В разметке стоит `<div ref={messagesEndRef} />` в конце списка сообщений. React кладёт настоящий DOM-элемент в `messagesEndRef.current`, и эффект прокручивает чат к нему при каждом новом сообщении.

б) Данные, которые меняются часто, но рисовать их не нужно

3D-герой обновляет наклон клавиатуры 60 раз в секунду. Если хранить координаты мыши в state, React перерисовывал бы весь герой 60 раз в секунду. Вместо этого координаты лежат в `useRef`, а стиль меняется напрямую у DOM-элемента:

`src/components/HeroExplodedParallax.jsx` · строки 146–166

```
const animate = () => {
  const ease = 0.08;
  current.current.x += (target.current.x - current.current.x) * ease;
  current.current.y += (target.current.y - current.current.y) * ease;
  current.current.scrollY += (target.current.scrollY - current.current.scrollY) * 0.1;

  const { x, y, scrollY: sy } = current.current;

  if (stage3dRef.current) {
    const rotX = 50 + y * 7;
    const rotZ = -26 + x * 8;
    const rotY = x * 5;
    stage3dRef.current.style.transform = `rotateX(${rotX.toFixed(2)}deg) rotateZ(${rotZ.toFixed(2)}deg) rotateY(${rotY.toFixed(2)}deg)`;
  }

  if (bgGlowRef.current) {
    bgGlowRef.current.style.transform = `translate3d(0, ${sy * 0.15}.toFixed(2)}px, 0)`;
  }

  animId = requestAnimationFrame(animate);
};
```

Это «плавное следование»: каждый кадр текущее значение приближается к целевому на 8% (`ease = 0.08`) — поэтому наклон мягкий, с инерцией.

5.5 useMemo — не пересчитывать зря

src/components/Catalog/Catalog.jsx · строки 66–75

```
const filteredProducts = useMemo(() => {
  const q = searchQuery.toLowerCase().trim();
  return PRODUCTS.filter((p) =>
    (!q || matchesQuery(p, q)) &&
    (selectedCategory === 'all' || p.category === selectedCategory) &&
    p.price <= priceLimit
  );
}, [selectedCategory, searchQuery, priceLimit]);

const sortedProducts = useMemo(() => [...filteredProducts].sort(SORTERS[sortBy]), [filteredProducts, sortBy]);
```

Каталог перерисовывается по многим причинам (наведение, загрузка). `useMemo` пересчитывает фильтр только если изменились категория, поиск или цена. Для 24 товаров выигрыш небольшой, но при тысячах товаров это важно.

5.6 useCallback

Каждый рендер создаёт новые функции. `useCallback(fn, [deps])` возвращает **ту же** функцию, пока зависимости не изменились. Нужен, когда функцию кладут в зависимости эффекта или передают в оптимизированные компоненты. В проекте — в `useTimedValue`, чтобы `show` была стабильной.

5.7 createContext + useContext

Корзина нужна десятку компонентов на разной глубине: шапке (счётчик), карточкам, странице товара, панели корзины, оформлению, тосту. Передавать её через props по всей цепочке («prop drilling») неудобно. Контекст решает это:

1. `const CartContext = createContext()` — создаём «канал».
2. `<CartContext.Provider value={...}>` — провайдер кладёт в канал данные и функции.
3. `useContext(CartContext)` — любой вложенный компонент читает из канала. В проекте это завернуто в удобный хук `useCart()`.

src/components/Catalog/ProductCard.jsx · строки 18–22

```
const { addToCart, productQty, decrementProduct } = useCart();
const { isFavorite, toggleFavorite } = useFavorites();
const favorited = isFavorite(product.id);
const inCart = productQty(product.id);
const isList = layout === 'list';
```

Цена удобства

Когда значение контекста меняется (например, добавили товар), перерисовываются **все** компоненты, которые вызывают `useCart()`. Для этого магазина это не проблема.

6 Собственные хуки проекта

Свой хук — это просто функция с именем `use...`, внутри которой вызываются другие хуки. Он позволяет вынести повторяющуюся логику из компонентов.

6.1 `usePersistentState` — состояние, которое переживает перезагрузку

`src/hooks/usePersistentState.js` · весь файл, 23 стр.

```
import { useState, useEffect } from 'react';

// useState, синхронизированный с localStorage (или sessionStorage)
export function usePersistentState(key, initial, storage = localStorage) {
  const [value, setValue] = useState(() => {
    try {
      const saved = storage.getItem(key);
      return saved ? JSON.parse(saved) : initial;
    } catch {
      return initial;
    }
  });

  useEffect(() => {
    try {
      storage.setItem(key, JSON.stringify(value));
    } catch (e) {
      console.error(`Ошибка сохранения ${key}:`, e);
    }
  }, [key, value, storage]);

  return [value, setValue];
}
```

Используется точно так же, как `useState`, но значение автоматически сохраняется в хранилище браузера:

- Первый рендер.** Функция внутри `useState(() => ...)` пытается прочесть значение по ключу. Хранилище хранит только строки, поэтому `JSON.parse` превращает строку обратно в массив/объект. Если там пусто или мусор — берём `initial`. `try/catch` защищает от ошибок (приватный режим, повреждённые данные).
- Каждое изменение.** `useEffect` с зависимостью `[value]` записывает `JSON.stringify(value)` обратно.
- Третий параметр** выбирает хранилище: по умолчанию `localStorage` (навсегда), для каталога — `sessionStorage` (до закрытия вкладки).

Где используется	Ключ	Хранилище
Корзина	<code>keycraft_retro_cart_v1</code>	<code>localStorage</code>
Применённый промокод	<code>keycraft_retro_promo_v1</code>	<code>localStorage</code>
Избранное	<code>keycraft_retro_favorites_v1</code>	<code>localStorage</code>
Настройки каталога	<code>keycraft_catalog_view</code>	<code>sessionStorage</code>

Суффикс `_v1` — версия формата. Если когда-нибудь поменять структуру корзины, можно сменить ключ на `_v2`, и старые несовместимые данные просто проигнорируются.

6.2 useTimedValue — значение «на время»

src/hooks/useTimedValue.js · весь файл, 17 стр.

```
import { useState, useEffect, useRef, useCallback } from 'react';

// Значение, которое само сбрасывается в null через `ms` миллисекунд
// (тосты, кнопки «Добавлено ✓» и т.п.)
export function useTimedValue(ms) {
  const [value, setValue] = useState(null);
  const timer = useRef();
  useEffect(() => () => clearTimeout(timer.current), []);

  const show = useCallback((next = true) => {
    setValue(next);
    clearTimeout(timer.current);
    timer.current = setTimeout(() => setValue(null), ms);
  }, [ms]);

  return [value, show];
}
```

Возвращает пару `[value, show]`. Вызов `show(x)` ставит значение `x` и через `ms` миллисекунд сам сбрасывает его в `null`. Детали:

- `timer = useRef()` хранит id таймера между рендерами. Если вызвать `show` повторно до истечения времени, старый таймер отменяется (`clearTimeout`) — отсчёт начинается заново, а не обрывается раньше.
- `useEffect(() => () => clearTimeout(...), [])` — эффект без тела, только с очисткой: при удалении компонента таймер отменяется, чтобы не менять состояние «мёртвого» компонента.
- `useCallback` — функция `show` одна и та же между рендерами.

Где	Вызов	Что показывает
CartContext	<code>useTimedValue(2500)</code>	Тост «добавлено в корзину»
ProductDetailPage	<code>useTimedValue(1800)</code>	Зелёная кнопка «Добавлено в корзину!»
FavoritesPage	<code>useTimedValue(2000)</code>	Текст «Все в корзине!»
ClackBotModal	<code>useTimedValue(1800)</code>	Галочка у товара, который только что добавили (хранит id товара)

6.3 Утилита rub()

src/utils/format.js · весь файл, 1 стр.

```
export const rub = (n) => `${n.toLocaleString('ru-RU')} ₽`;
```

Не хук, а обычная функция: `toLocaleString('ru-RU')` расставляет пробелы между разрядами:

`rub(18900)` → `"18 900 ₽"`.

7 Где хранится состояние

В проекте данные живут на **пяти уровнях**. Главный вопрос для каждого значения: «что с ним будет, если перезагрузить страницу?»

1. Код (src/data/products.js)
Товары, категории, промо, промокоды. Не меняются во время работы. Меняются только правкой файла.

навсегда, одинаково у всех

2. localStorage
Корзина, промокод, избранное, история заказов. Хранится в браузере пользователя.

переживает перезагрузку и закрытие браузера

3. sessionStorage
Сортировка, цена, страница и вид каталога.

до закрытия вкладки

4. Адресная строка (location.hash)
#home, #catalog, #favorites, #product/kb-lumina-75 — какая страница открыта.

переживает перезагрузку, можно отправить ссылкой

5. Память React: useState / useRef / контекст
Открыта ли корзина, вкладка, количество, поиск, категория, форма заказа, сообщения бота, позиции прокрутки... Всё, что не сохранено явно.

исчезает при перезагрузке

Рис. 2. Пять уровней хранения данных

7.1 Полная карта состояния

Обозначения: **state** — useState, **ref** — useRef, **local** — localStorage, **session** — sessionStorage, **url** — адресная строка.

Значение	Где объявлено	Хранение	Кто меняет / зачем
Навигация — MainShop (App.jsx)			
currentPage	MainShop	state url	Какая страница показана: home / catalog / product / favorites. Дублируется в hash.
activeProductId	MainShop	state url	id открытого товара (часть hash #product/...)
selectedCategory	MainShop	state	Категория каталога; меняют главная, футер, фильтры, крошки
searchQuery	MainShop	state	Поиск; общий для шапки (десктоп) и каталога (мобилка)
isChatBotOpen	MainShop	state	Открыт ли чат-бот
location, scrollMemory, pendingScroll	MainShop	ref	Где мы сейчас, запомненные позиции прокрутки, куда прокрутить после смены страницы
Корзина — CartContext			
cartItems	CartProvider	state local	Позиции корзины (формат — раздел 8)

<code>appliedPromo</code>	CartProvider	<code>state</code> <code>local</code>	Применённый промокод <code>{code, discountPercent, description}</code>
<code>promoError</code>	CartProvider	<code>state</code>	Текст ошибки «Неверный промокод»
<code>isCartOpen</code> , <code>isCheckoutOpen</code>	CartProvider	<code>state</code>	Открыты ли панель корзины и окно оформления
<code>toast</code>	CartProvider	<code>state</code> (2.5 c)	Текущее уведомление <code>{text, id}</code>
заказы	<code>createOrder()</code>	<code>local</code>	Ключ <code>keycraft_retro_orders_v1</code> ; только записываются, в интерфейсе нигде не читаются
Избранное — FavoritesContext			
<code>favorites</code>	FavoritesProvider	<code>state</code> <code>local</code>	Массив id товаров: <code>["kb-lumina-75", "sw-oil-king"]</code>
Каталог			
<code>view</code> = <code>sortBy</code> , <code>priceLimit</code> , <code>currentPage</code> , <code>itemsPerPage</code> , <code>viewMode</code>	Catalog	<code>state</code> <code>session</code>	Чтобы после товара вернуться к той же странице/сортировке
<code>isLoading</code>	Catalog	<code>state</code>	Показывать «скелетоны» 220 мс после смены фильтров
<code>prevFilterKey</code>	Catalog	<code>ref</code>	Прошлые фильтры — чтобы понять, что они действительно поменялись
Остальные компоненты			
<code>selectedOption</code> , <code>quantity</code> , <code>activeTab</code> , <code>isAddedRecently</code>	ProductDetailPage	<code>state</code>	Выбранный вариант, количество, вкладка, «Добавлено!»
<code>viewMode</code> , <code>allAdded</code>	FavoritesPage	<code>state</code>	Сетка/список, «Все в корзине!»
<code>promoInput</code>	CartDrawer	<code>state</code>	Текст в поле промокода
<code>formData</code> , <code>errors</code> , <code>completedOrder</code>	CheckoutModal	<code>state</code>	Поля формы, ошибки валидации, готовый заказ (для чека)
<code>messages</code> , <code>inputVal</code> , <code>addedId</code>	ClackBotModal	<code>state</code>	История чата (живёт, пока открыта вкладка)
<code>currentIndex</code> , <code>isPaused</code>	PromoCarousel	<code>state</code>	Номер слайда, пауза при наведении
<code>explosion</code> , <code>activeLayerId</code> , <code>isolateActiveLayer</code>	HeroExplodedParallax	<code>state</code>	Степень разнесения слоёв 0–1, выбранный слой, режим изоляции
<code>target</code> , <code>current</code> , 3 DOM-ref	HeroExplodedParallax	<code>ref</code>	Координаты мыши/прокрутки для анимации 60 fps

7.2 Как посмотреть хранилище своими глазами

Откройте сайт → `F12` → вкладка **Application** (в Firefox — **Storage**) → **Local Storage / Session Storage** → адрес сайта. Там видны ключи из таблицы и их JSON. Можно удалить ключ — после перезагрузки

корзина/избранное станут пустыми.

7.3 Почему одни данные сохраняются, а другие нет

- **Корзина и избранное** — пользователь ожидает их увидеть завтра → `localStorage`.
- **Настройки каталога** — полезны в рамках одного визита, но завтра логичнее начать с начала → `sessionStorage`.
- **Открыта ли корзина, вкладка, форма** — временные «состояния интерфейса» → обычный `state`.
- **Позиции прокрутки и координаты мыши** — нужны, но не влияют на разметку → `useRef` (без перерисовок).
- **Текущая страница** — должна работать кнопка «Назад» и ссылка «поделиться» → адресная строка.

8 Глобальное состояние: контексты

8.1 CartContext — сердце магазина

Файл `src/context/CartContext.jsx`. Хранит корзину и всё, что с ней связано, и отдаёт это любому компоненту через `useCart()`.

Формат позиции корзины

```
{
  id: "kb-lumina-75-Boba U4T (Tactile)", // id товара + выбранный вариант
  product: { id, title, price, image, ... }, // ПОЛНАЯ копия товара из products.js
  quantity: 2,
  selectedOption: "Boba U4T (Tactile)",
  addedAt: 1759406400000 // время добавления (мс)
}
```

Один и тот же товар с разными вариантами — это **разные позиции** (разные id). Поэтому есть `productQty(productId)`, который суммирует все варианты товара — его показывает счётчик на карточке.

Состояние провайдера

`src/context/CartContext.jsx` · строки 6–25

```
const CartContext = createContext();

const FREE_DELIVERY_FROM = 5000;
const DELIVERY_PRICE = 350;

const defaultOption = (product) => product.options?.[0] ?? 'Стандарт';

export function CartProvider({ children }) {
  const [cartItems, setCartItems] = usePersistentState('keycraft_retro_cart_v1', []);
  const [appliedPromo, setAppliedPromo] = usePersistentState('keycraft_retro_promo_v1', null);
  const [promoError, setPromoError] = useState('');
  const [isCartOpen, setIsCartOpen] = useState(false);
  const [isCheckoutOpen, setIsCheckoutOpen] = useState(false);
  const [toast, setToast] = useTimedValue(2500);

  // Тост «добавлено» — только когда корзина закрыта (иначе всё и так видно)
  const showToast = (text) => {
    if (!isCartOpen) setToast({ text, id: Date.now() });
  };
  const hideToast = () => setToast(null);
```

Добавление в корзину

src/context/CartContext.jsx · строки 27–38

```
// silent: место вызова само показывает результат (счётчик на карточке и т.п.)
const addToCart = (product, quantity = 1, selectedOption = null, { silent = false } = {}) => {
  const opt = selectedOption || defaultOption(product);
  const id = `${product.id}-${opt}`;

  setCartItems((prev) =>
    prev.some((item) => item.id === id)
      ? prev.map((item) => (item.id === id ? { ...item, quantity: item.quantity + quantity } : item))
      : [...prev, { id, product, quantity, selectedOption: opt, addedAt: Date.now() }]
  );
  if (!silent) showToast(`«${product.title}» добавлен в корзину`);
};
```

- Если вариант не выбран — берётся первый из `product.options` (или «Стандарт»).
- Если такая позиция уже есть — увеличиваем количество (через `map`, создавая новый объект); иначе — добавляем новую в конец массива.
- Параметр `{ silent: true }` отключает тост: карточка, страница товара, избранное и чат-бот сами показывают результат.

Производные значения — считаются, а не хранятся

src/context/CartContext.jsx · строки 83–88

```
const totalItems = cartItems.reduce((acc, item) => acc + item.quantity, 0);
const subtotal = cartItems.reduce((acc, item) => acc + item.product.price * item.quantity, 0);
const discountAmount = appliedPromo ? Math.round((subtotal * appliedPromo.discountPercent) / 100) : 0;
const totalPrice = Math.max(0, subtotal - discountAmount);
const deliveryCost = subtotal >= FREE_DELIVERY_FROM ? 0 : DELIVERY_PRICE;
const freeDeliveryLeft = Math.max(0, FREE_DELIVERY_FROM - subtotal);
```

Важный принцип: не храните то, что можно вычислить

Сумма, скидка, доставка не лежат в state — они пересчитываются при каждом рендере из `cartItems` и `appliedPromo`. Поэтому они **никогда не рассинхронизируются**. Например, при сумме 4 500 ₽: доставка 350 ₽, до бесплатной осталось 500 ₽; с промокодом ТНОСК20 скидка = $\text{round}(4500 \times 20 / 100) = 900$ ₽, итого $3\,600 + 350 = 3\,950$ ₽.

Все функции и значения контекста

Имя	Что делает
<code>cartItems</code>	Массив позиций
<code>addToCart(product, qty, option, {silent})</code>	Добавить товар (см. выше)
<code>updateQuantity(itemId, ±1)</code>	Изменить количество позиции; при 0 позиция удаляется (<code>filter</code>)
<code>removeFromCart(itemId)</code>	Удалить позицию
<code>clearCart()</code>	Очистить корзину и промокод
<code>productQty(productId)</code>	Сколько штук товара (по всем вариантам)
<code>decrementProduct(productId)</code>	Минус 1 с последней добавленной позиции товара (кнопка «−» на карточке)

<code>applyPromo(code) / removePromo()</code>	Проверить код по <code>PROMOCODES</code> (регистр не важен), применить или убрать
<code>totalItems, subtotal, discountAmount, totalPrice, deliveryCost, freeDeliveryLeft</code>	Вычисляемые итоги
<code>isCartOpen / setIsCartOpen , isCheckoutOpen / setIsCheckoutOpen</code>	Открытие панели корзины и окна заказа
<code>createOrder(customer)</code>	Собрать заказ (номер KC-XXXXXXX, дата, позиции, суммы), дописать в <code>localStorage</code> , очистить корзину, вернуть заказ для чека
<code>toast, showToast, hideToast</code>	Уведомление; <code>showToast</code> ничего не делает, если корзина уже открыта

8.2 FavoritesContext

`src/context/FavoritesContext.jsx` · весь файл, 27 стр.

```
import { createContext, useContext } from 'react';
import { usePersistentState } from '../hooks/usePersistentState';

const FavoritesContext = createContext();

// Без тостов: результат и так виден (сердце на карточке + счётчик в меню)
export function FavoritesProvider({ children }) {
  const [favorites, setFavorites] = usePersistentState('keycraft_retro_favorites_v1', []);

  const isFavorite = (productId) => favorites.includes(productId);

  const toggleFavorite = (productId) => {
    setFavorites((prev) => (prev.includes(productId) ? prev.filter((id) => id !== productId) :
[...prev, productId]));
  };

  const clearFavorites = () => setFavorites([]);

  return (
    <FavoritesContext.Provider value={{ favorites, favoritesCount: favorites.length, toggleFavorite,
isFavorite, clearFavorites }}>
      {children}
    </FavoritesContext.Provider>
  );
}

export function useFavorites() {
  return useContext(FavoritesContext);
}
```

Хранит только `id` товаров, а не целые объекты, — так данные в `localStorage` всегда совпадают с актуальным каталогом. Страница избранного «восстанавливает» товары: `PRODUCTS.filter(p => favorites.includes(p.id))`.

9 Данные магазина: products.js

Вместо базы данных — один файл с четырьмя экспортами. Компоненты импортируют их напрямую:

```
import { PRODUCTS } from '../data/products'.
```

9.1 CATEGORIES — 9 категорий

src/data/products.js · строки 1–11

```
export const CATEGORIES = [
  { id: 'all', name: 'Все товары', icon: 'Sparkles', count: 24 },
  { id: 'keyboards', name: 'Готовые клавиатуры', icon: 'Keyboard', count: 4 },
  { id: 'switches', name: 'Переключатели (Свитчи)', icon: 'Cpu', count: 5 },
  { id: 'keycaps', name: 'Наборы кейкапов', icon: 'Grid', count: 4 },
  { id: 'modding', name: 'Шумоизоляция и моддинг', icon: 'Layers', count: 3 },
  { id: 'tools', name: 'Смазка и инструмент', icon: 'Wrench', count: 3 },
  { id: 'deskpads', name: 'Коврики (Дескпады)', icon: 'Square', count: 2 },
  { id: 'cables', name: 'Авиатор-кабели', icon: 'Cable', count: 2 },
  { id: 'wristrests', name: 'Подставки под запястья', icon: 'Maximize2', count: 1 },
];
```

Счётчики записаны вручную

Поле `count` не вычисляется: если добавить товар, число в фильтре не изменится само (см. раздел 16).

9.2 PRODUCTS — 24 товара

Распределение: клавиатуры 4, свитчи 5, кейкапы 4, шумоизоляция 3, инструменты 3, коврики 2, кабели 2, подставки 1. Формат одного товара:

src/data/products.js · строки 13–36

```
export const PRODUCTS = [
  {
    id: 'kb-lumina-75',
    title: 'KeyCraft Retro 75 «Walnut & Brass»',
    category: 'keyboards',
    categoryName: 'Готовые клавиатуры',
    price: 18900,
    oldPrice: 21500,
    rating: 4.9,
    reviewsCount: 42,
    inStock: true,
    badge: 'Выбор мастера',
    image: '/images/kb-lumina-75.svg',
    description: 'Флагманский кастом в корпусе из массива американского ореха с латунным плейтом, прокладками Gasket Mount и двойной Рогоп-шумоизоляцией. Звучание глубокое, теплое и благородное.',
    specs: {
      'Форм-фактор': '75% (82 клавиши + роллер громкости)',
      'Материал корпуса': 'Американский орех + масляная пропитка',
      'Крепление плейта': 'Gasket Mount (силиконовые демпферы)',
      'Подключение': 'Tri-mode: Bluetooth 5.2 / 2.4Ghz / Type-C',
      'Подсветка': 'Теплый янтарный бекграунд',
      'Аккумулятор': '4000 мАч (до 4 недель работы)',
    },
    options: ['Gateron Oil King (Thock)', 'Boba U4T (Tactile)', 'Kailh Box Silent']
  },
];
```

Поле	Смысл
<code>id</code>	Уникальный ключ; используется в URL (<code>#product/kb-lumina-75</code>), избранном и корзине
<code>category / categoryName</code>	id категории для фильтра и её название для показа
<code>price / oldPrice</code>	Цена и старая цена (если есть — показывается зачёркнутой и «Экономия»)
<code>rating , reviewsCount</code>	Для показа и сортировки «по рейтингу» / «по популярности»
<code>badge</code>	Плашка на фото («Хит продаж»)
<code>image</code>	Путь к картинке из <code>public/</code>
<code>specs</code>	Объект «характеристика → значение»; выводится через <code>Object.entries</code>
<code>options</code>	Варианты исполнения; первый выбирается по умолчанию
<code>inStock</code>	Есть у всех товаров, но в интерфейсе пока не используется (везде «В наличии»)

9.3 PROMO_SLIDES и PROMOCODES

`src/data/products.js` · строки 553–557

```
export const PROMOCODES = {
  'VINTAGE10': { discountPercent: 10, description: 'Скидка 10% на первый заказ' },
  'THOCK20': { discountPercent: 20, description: 'Скидка 20% для ценителей глубокого звука' },
  'CLACK5': { discountPercent: 5, description: 'Скидка 5% на аксессуары' }
};
```

Слайды карусели ссылаются на товар через `productId`. Промокоды — объект «код → скидка». Проверка: `PROMOCODES[code.trim().toUpperCase()]`.

Промокоды видны всем

Так как всё работает в браузере, любой может открыть код сайта и увидеть все промокоды и их скидки. В настоящем магазине проверку делает сервер.

10 Навигация без библиотек

Обычно в React используют React Router. Здесь роутинг написан вручную на **hash** — части адреса после **#**. Её изменение не перезагружает страницу, но попадает в историю браузера, поэтому кнопки «Назад/Вперёд» работают.

Адрес	Что показано
<code>/#home</code> или пусто	Главная
<code>/#catalog</code>	Каталог
<code>/#favorites</code>	Избранное
<code>/#product/sw-oil-king</code>	Страница товара с id <code>sw-oil-king</code>

10.1 Ядро: функция go

src/App.jsx · строки 29–46

```
// Прокрутка: запоминаем позицию каждой страницы, чтобы при возврате
// (кнопка «Назад» браузера или «Вернуться в каталог») оказаться там же
const location = useRef({ page: 'home', productId: null });
const scrollMemory = useRef({});
const pendingScroll = useRef(null);
const keyOf = ({ page, productId }) => (page === 'product' ? `product/${productId}` : page);

// Возвращает false, если мы уже на этой странице
const go = (page, productId = null, { restore = false } = {}) => {
  const next = { page, productId: page === 'product' ? productId : null };
  if (keyOf(next) === keyOf(location.current)) return false;
  scrollMemory.current[keyOf(location.current)] = window.scrollY;
  pendingScroll.current = restore ? scrollMemory.current[keyOf(next)] ?? 0 : 0;
  location.current = next;
  setCurrentPage(page);
  if (next.productId) setActiveProductId(next.productId);
  return true;
};
```

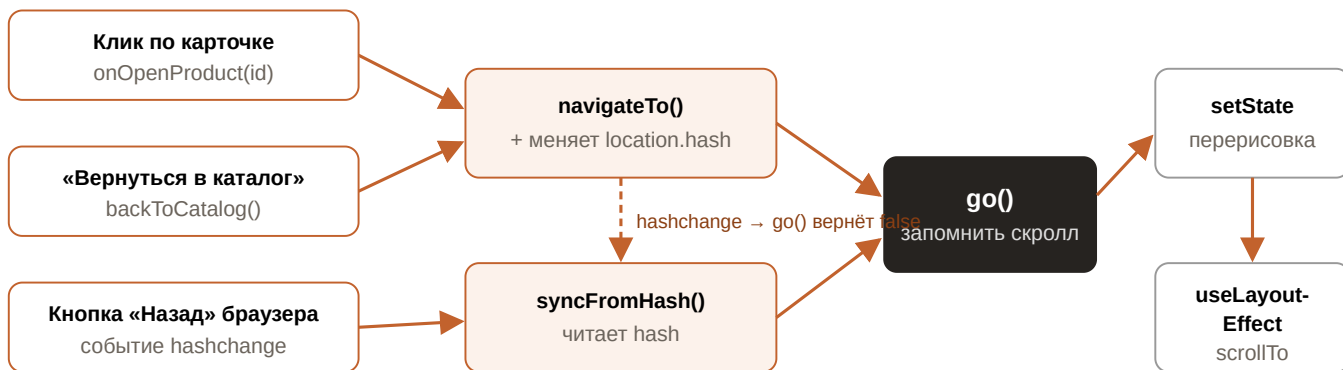
go — единственное место, где меняется текущая страница. Перед уходом она запоминает прокрутку старой страницы, решает, куда прокрутить новую (в начало или на запомненное место), и обновляет state.

10.2 navigateTo и удобные обёртки

src/App.jsx · строки 67–75

```
const navigateTo = (page, productId = null, category = null, { restore = false } = {}) => {
  if (category) setSelectedCategory(category);
  const changed = go(page, productId, { restore: restore && !category });
  if (changed) window.location.hash = productId ? `product/${productId}` : page;
  else window.scrollTo({ top: 0, behavior: 'smooth' }); // повторный тап по текущему разделу — вверх
};

const openProduct = (id) => navigateTo('product', id);
const openCatalog = (category = null) => navigateTo('catalog', null, category);
const backToCatalog = () => navigateTo('catalog', null, null, { restore: true });
```



Из-за проверки «уже на этой странице» изменение hash самим navigateTo не вызывает повторную навигацию.

Рис. 3. Три способа сменить страницу сходятся в функции go()

10.3 Как выбирается страница для показа

В разметке `MainShop` страницы рендерятся условно — в каждый момент существует только одна:

`src/App.jsx` · строки 205–212

```

{currentPage === 'product' && (
  <ProductDetailPage
    key={activeProductId}
    productId={activeProductId}
    onBackToCatalog={backToCatalog}
    onOpenProduct={openProduct}
  />
)}

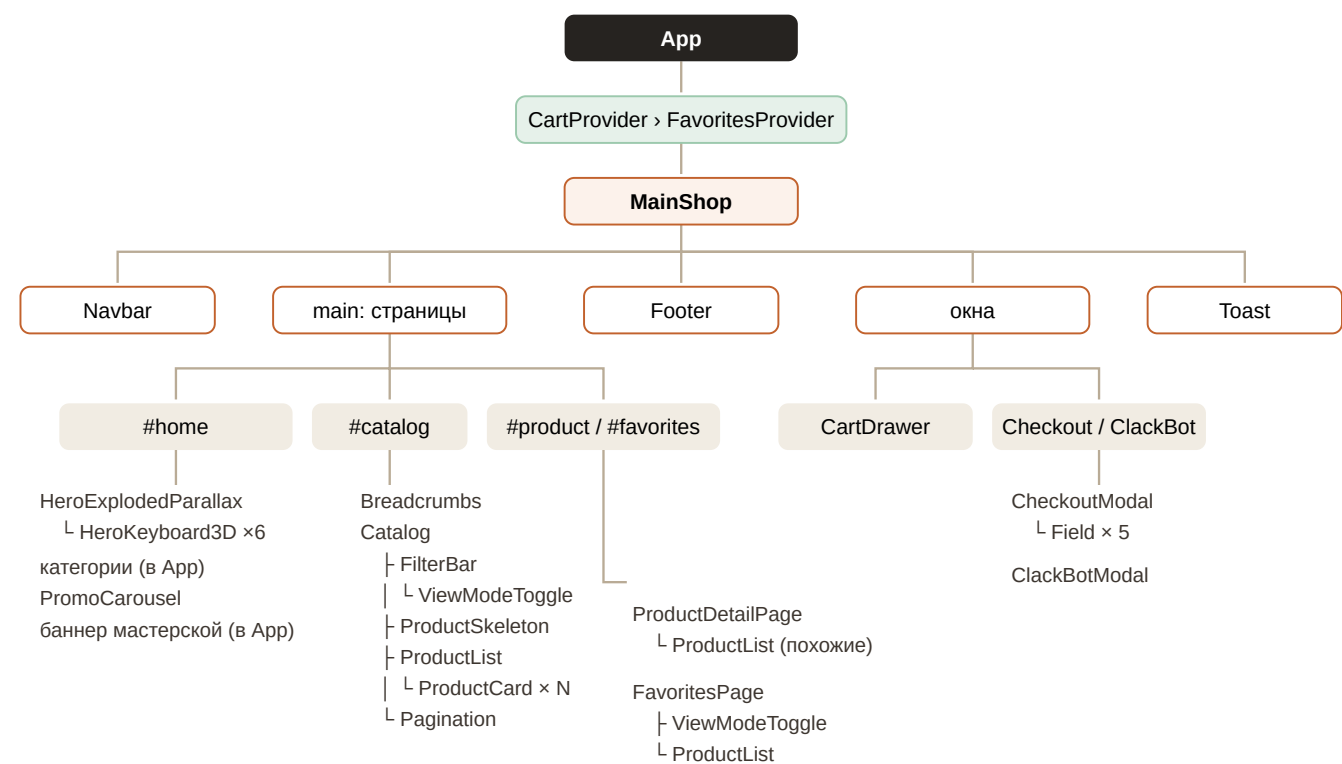
```

Трюк с key

`key={activeProductId}` говорит React: «для другого товара — это другой компонент». При переходе на похожий товар старая страница удаляется, новая создаётся с чистым state: сбрасываются выбранный вариант, количество и вкладка. Без `key` React переиспользовал бы компонент, и на новом товаре оставался бы вариант от старого.

Окна (корзина, оформление, чат-бот, тост) рендерятся **всегда**, но сами возвращают `null`, когда закрыты. Поэтому они не зависят от текущей страницы.

11 Дерево компонентов



Navbar внутри: SearchInput (десктоп), NAV_ITEMS × 2 (шапка и нижняя мобильная панель), MobileBadge.
Компоненты, использующие useCart(): Navbar, PromoCarousel, ProductCard, ProductDetailPage, FavoritesPage, CartDrawer, CheckoutModal, ClackBotModal, Toast. useFavorites(): Navbar, ProductCard, ProductDetailPage, FavoritesPage.

Рис. 4. Кто внутри кого рендерится

12 Разбор компонентов по файлам

12.1 App.jsx → MainShop

Кроме навигации (раздел 10) рендерит главную страницу прямо внутри себя: 3D-героя, быстрые категории (кнопки из `CATEGORIES` без «Все товары»), карусель и тёмный баннер «Философия мастерской». Обратите внимание на маленькие функции-обёртки: `openProduct`, `openCatalog`, `backToCatalog`, `openChatBot` — они делают разметку короче и понятнее.

12.2 Navbar.jsx

- Своего state нет. Читает `totalItems` из корзины и `favoritesCount` из избранного.
- **Пункты меню** — массив `NAV_ITEMS`. Один и тот же массив рисуется дважды: в шапке (десктоп, `hidden md:flex`) и в нижней панели (мобилка, `md:hidden`).
- Поиск на десктопе — поле в шапке; при вводе текста не на странице каталога выполняется переход в каталог. На мобилке — кнопка-лупа: переход в каталог и `focus()` поля `#mobile-search`.
- Счётчик корзины «подпрыгивает»: у бейджа `key={totalItems}` — при изменении числа React создаёт новый элемент, и CSS-анимация `animate-pop` проигрывается заново.

`src/components/Navbar.jsx` · строки 5–9

```
const NAV_ITEMS = [  
  { page: 'home', label: 'Главная', Icon: Home },  
  { page: 'catalog', label: 'Каталог', Icon: Grid },  
  { page: 'favorites', label: 'Избранное', Icon: Heart },  
];
```

12.3 HeroExplodedParallax.jsx + HeroKeyboard3D.jsx — 3D-клавиатура

Самая «зрелищная» часть. Никакого WebGL — только CSS 3D-трансформации и SVG.

1. **Данные слоёв** — массив `KEYBOARD_LAYERS`: для каждого слоя название, описание, высота «в сборе» (`assembledZ`) и «в разборе» (`explodedZ`) и SVG-компонент.
2. **Сцена** — div со стилем `perspective: 1400px`, внутри — «стол» с `transform-style: preserve-3d` и поворотом `rotateX(50deg) rotateZ(-26deg)`. Каждый слой — абсолютный div, поднятый по оси Z.
3. **Высота слоя** = `assembledZ + (explodedZ - assembledZ) × explosion`, плюс 50px, если слой выбран. `explosion` — значение ползунка от 0 до 1.
4. **Анимация наклона** — цикл `requestAnimationFrame` в `useEffect` + координаты в `useRef` (раздел 5.4). Очистка эффекта отменяет кадр и снимает обработчики.
5. **SVG-слои** генерируются из данных. Например, клавиши кейкапов описаны таблицей рядов:

```
// ряд обычных клавиш шириной 23.5 с шагом 27
const letters = (x0, chars) => [...chars].map((c, i) => [x0 + i * 27, 23.5, 'top', c]);

// ряды: y, высота, базовая линия текста, клавиши [x, ширина, стиль, подпись]
const KEY_ROWS = [
  {
    y: 10, h: 18, ty: 22,
    keys: [
      ...[44, 148, 252].flatMap((x0, g) => [0, 1, 2, 3].map((i) => [x0 + i * 25, 22, 'fn', `F${g * 4 + i + 1}`])),
      [356, 22, 'sageSm', 'PS'], [381, 22, 'sageSm', 'SL'], [406, 22, 'sageSm', 'PAU'], [431, 30, 'accent', 'DEL'],
    ],
  },
  {
    y: 33, h: 19, ty: 46,
    keys: [...letters(14, '~1234567890-='), [365, 45, 'mod', 'BACKSPACE'], [414, 28, 'sage', 'HOME']],
  }
];
```

Компонент `KeyRow` превращает каждую запись `[x, ширина, стиль, подпись]` в прямоугольник и текст. Функция `outline(isHighlighted, цвет)` даёт выбранному слою терракотовую обводку.

12.4 PromoCarousel.jsx

State: `currentIndex` и `isPaused`. Переключение по кругу — остаток от деления: `(prev + delta + SLIDES) % SLIDES` (прибавление `SLIDES` защищает от отрицательных чисел при шаге `-1`). Автопрокрутка — эффект с `setInterval` (раздел 5.2). Связанный товар ищется через `PRODUCTS.find(p => p.id === slide.productId)`.

12.5 Catalog — конвейер данных



Рис. 5. Как 24 товара превращаются в страницу каталога

```
const SORTERS = {
  popular: (a, b) => b.reviewsCount - a.reviewsCount,
  'price-asc': (a, b) => a.price - b.price,
  'price-desc': (a, b) => b.price - a.price,
  rating: (a, b) => b.rating - a.rating,
};

const matchesQuery = (p, q) =>
  [p.title, p.description, p.categoryName].some((field) => field.toLowerCase().includes(q));
```

Сортировки — объект функций-компараторов. `sort` вызывается на **копии** массива `( [...filteredProducts] )`, потому что `sort` меняет массив на месте.

Сохранение настроек и сброс страницы

src/components/Catalog/Catalog.jsx · строки 45–64

```
// Настройки каталога переживают переход на товар и обратно (в пределах вкладки)
const [view, setView] = usePersistentState('keycraft_catalog_view', DEFAULT_VIEW, sessionStorage);
const { sortBy, priceLimit, currentPage, itemsPerPage, viewMode } = view;
const set = (key) => (value) => setView((v) => ({ ...v, [key]: value }));
const [setSortBy, setPriceLimit, setCurrentPage, setItemsPerPage, setViewMode] =
  ['sortBy', 'priceLimit', 'currentPage', 'itemsPerPage', 'viewMode'].map(set);
const [isLoading, setIsLoading] = useState(false);

// При смене фильтров — на первую страницу и короткий «скелетон».
// При возврате в каталог фильтры те же, поэтому страница сохраняется.
const filterKey = JSON.stringify([selectedCategory, searchQuery, sortBy, priceLimit]);
const prevFilterKey = useRef(filterKey);
useEffect(() => {
  if (prevFilterKey.current === filterKey) return;
  prevFilterKey.current = filterKey;
  setView((v) => ({ ...v, currentPage: 1 }));
  setIsLoading(true);
  const timer = setTimeout(() => setIsLoading(false), 220);
  return () => clearTimeout(timer);
}, [filterKey, setView]);
```

- Пять настроек объединены в один объект `view` в `sessionStorage`. Функция `set(key)` создаёт «сеттер» для одного поля — так получаются привычные `setSortBy`, `setCurrentPage` и т.д.
- `filterKey` — все фильтры одной строкой. `useRef` помнит прошлую строку. При первом показе (возврат со страницы товара) строки совпадают → страница **не** сбрасывается. При реальной смене фильтра — на 1-ю страницу и 220 мс «скелетон».

12.6 FilterBar.jsx и Pagination.jsx

Оба без `state` — «контролируемые». `FilterBar` рисует ленту категорий (горизонтальный скролл, класс `no-scrollbar`), ползунок цены, выбор сортировки, переключатель вида и кнопку «Сброс» (видна, только если фильтры активны). `ViewModeToggle` экспортируется и переиспользуется на странице избранного. `Pagination` строит кнопки страниц через `Array.from({ length: totalPages })` и после клика плавно прокручивает к началу каталога (`scrollIntoView`). Если страница одна — возвращает `null`.

12.7 ProductCard.jsx — карточка товара

Своего `state` нет: всё берёт из контекстов. Два вида — `grid` и `list`; общие куски (картинка, рейтинг, цена, кнопка) вынесены в переменные с JSX, чтобы не дублировать.

src/components/Catalog/ProductCard.jsx · строки 24–30

```
// Кнопки внутри карточки не должны открывать страницу товара
const stop = (fn) => (e) => {
  e.stopPropagation();
  fn();
};
const add = stop(() => addToCart(product, 1, null, { silent: true }));
const handleToggleFav = stop(() => toggleFavorite(product.id));
```

Вся карточка кликабельна (открывает товар). Чтобы клик по «сердечку» или «+» не открывал товар, вызывается `e.stopPropagation()` — событие не «всплывает» до карточки. Функция `stop(fn)` — маленькая фабрика, которая оборачивает действие в такой обработчик.

```
// До добавления — кнопка «В корзину», после — счётчик «– N +»
const cartButton = inCart === 0 ? (
  <button
    onClick={add}
    className={`btn-retro text-xs ${isList ? 'py-2 px-3.5' : 'py-1.5 px-3'} flex items-center gap-1.5
      transition-all shadow-sm bg-stone-900 text-white hover:bg-stone-800`}
    aria-label="Добавить в корзину"
  >
    <ShoppingBag className="w-3.5 h-3.5" />
    <span>В корзину</span>
  </button>
) : (
```

Когда `productQty` больше нуля, вместо кнопки показывается счётчик «– N шт. +», а карточка получает рамку цвета акцента. Компонент `ProductList` из этого же файла рисует сетку или список карточек — используется в каталоге, избранном и «похожих товарах».

12.8 ProductDetailPage.jsx — страница товара

- Находит товар по id (если не нашёл — первый товар, чтобы не упасть на битой ссылке).
- State: вариант (`selectedOption`), количество, вкладка, флаг «Добавлено!».
- Статические тексты вынесены в константы над компонентом: `VALUE_BADGES`, `BOX_CONTENTS`, `SAMPLE_REVIEWS`; вкладки — массив `tabs`.
- «Похожие товары»: сначала той же категории, затем остальные, первые три.
- Если товар уже в корзине — плашка «В корзине уже N шт. · Открыть корзину».

```
// Сначала товары той же категории, затем остальные — всего 3
const others = PRODUCTS.filter(p => p.id !== product.id);
const relatedProducts = [
  ...others.filter(p => p.category === product.category),
  ...others.filter(p => p.category !== product.category),
].slice(0, 3);
```

12.9 FavoritesPage.jsx

Собирает товары по id из избранного, даёт переключить вид, «Добавить все в корзину» (каждый товар добавляется «тихо», а кнопка на 2 с меняет текст) и очистить список. Пустое состояние — отдельная разметка с кнопкой «Перейти в каталог».

12.10 CartDrawer.jsx — панель корзины

- На мобилке — во весь экран (`w-full`), с ширины 640px — 448px справа (`sm:max-w-md`).
- Эффект: `Esc` закрывает, прокрутка страницы под панелью блокируется (`body.style.overflow = 'hidden'`), очистка всё возвращает.
- Клик по фото/названию — закрыть панель и открыть товар (функция `goTo(fn)`).
- Прогресс-бар до бесплатной доставки: ширина = `subtotal / (subtotal + freeDeliveryLeft) × 100%`.
- Промокод — форма с `onSubmit`; `e.preventDefault()` отменяет стандартную отправку формы (перезагрузку страницы).

12.11 CheckoutModal.jsx — оформление заказа

src/components/CheckoutModal.jsx · строки 9–17

```
function Field({ label, error, type = 'text', ...props }) {
  return (
    <div>
      <label className="block text-xs font-mono text-stone-600 mb-1">{label}</label>
      <input type={type} {...props} className={` ${inputClass} ${error ? 'border-red-500' : 'border-stone-300'} `} />
      {error && <p className="text-[10px] text-red-500 font-mono mt-0.5">{error}</p>}
    </div>
  );
}
```

`Field` — переиспользуемое поле с подписью и ошибкой. Функция `field(name)` возвращает для него `value`, `error` и `onChange`, поэтому каждое поле — одна строка: `<Field {...field('phone')} label="Телефон *" type="tel" />`. Синтаксис `{...obj}` раскладывает объект в атрибуты.

src/components/CheckoutModal.jsx · строки 50–65

```
const validate = () => {
  const errs = {};
  if (!formData.fullName.trim()) errs.fullName = 'Укажите ваше имя';
  if (!formData.phone.trim() || formData.phone.length < 7) errs.phone = 'Укажите телефон';
  if (!formData.email.trim() || !formData.email.includes('@')) errs.email = 'Укажите email';
  if (!formData.address.trim()) errs.address = 'Укажите адрес доставки';
  setErrors(errs);
  return Object.keys(errs).length === 0;
};

const handleSubmit = (e) => {
  e.preventDefault();
  if (!validate()) return;

  setCompletedOrder(createOrder({ ...formData, deliveryCost: delivery }));
  confetti({ particleCount: 50, spread: 60, origin: { y: 0.6 }, colors: ['#C2622D', '#D97706', '#262320'] });
};
```

Валидация простая: имя и адрес не пустые, телефон не короче 7 символов, в email есть «@». Ошибки кладутся в state `errors` и показываются под полями. После успешной отправки `createOrder` возвращает заказ — он сохраняется в `completedOrder`, и окно переключается в режим «чек». Самовывоз обнуляет доставку: `const delivery = formData.deliveryMethod === 'pickup' ? 0 : deliveryCost`.

12.12 ClackBotModal.jsx — чат-бот на правилах

src/components/ClackBotModal.jsx · строки 55–70

```
function botReply(query) {
  const q = query.toLowerCase();
  const rule = RULES.find((r) => r.keys.some((k) => q.includes(k)));
  if (rule) return { text: rule.text, products: byIds(...rule.products), options: rule.options };

  const matches = PRODUCTS.filter((p) =>
    [p.title, p.categoryName, p.description].some((f) => f.toLowerCase().includes(q))
  ).slice(0, 2);
  return matches.length > 0
    ? { text: 'Вот подходящие товары по вашему запросу:', products: matches, options: [] }
    : {
        text: 'Я могу помочь с выбором переключателей по звуку, посоветовать клавиатуру или подсказать промокод.',
        products: [],
        options: [FIRST_BUILD, QUIET, THOCK],
      };
}
```

Сценарии лежат в массиве `RULES`: список ключевых слов, текст ответа, id товаров и кнопки-подсказки. `find` + `some` находят первое правило, где в вопросе встречается любое ключевое слово. Если правил нет — ищем товары по тексту, если и их нет — общий ответ. Ответ бота добавляется через `setTimeout(..., 300)`, чтобы выглядело так, будто бот «думает». История живёт в state компонента — после перезагрузки страницы чат начинается заново.

12.13 Toast.jsx — уведомление

src/components/Toast.jsx · строки 6–15

```
export function Toast() {
  const { toast, hideToast, isCartOpen, isCheckoutOpen, setIsCartOpen } = useCart();
  if (!toast || isCartOpen || isCheckoutOpen) return null;

  const openCart = () => {
    hideToast();
    setIsCartOpen(true);
  };

  return (
```

Не показывается, если открыта корзина или оформление. `key={toast.id}` на корневом элементе: новое уведомление = новый элемент, анимация появления `animate-toast-in` проигрывается снова.

12.14 Breadcrumbs.jsx и Footer.jsx

Простые компоненты без state. В футере преимущества, ссылки на категории и строки «Информация» вынесены в массивы `FEATURES`, `LINKS`, `INFO` и выводятся через `map`. Год в копирайте — `new Date().getFullYear()`, поэтому обновляется сам.

13 Сценарии: что происходит при действиях

13.1 «Нажал + на карточке товара»

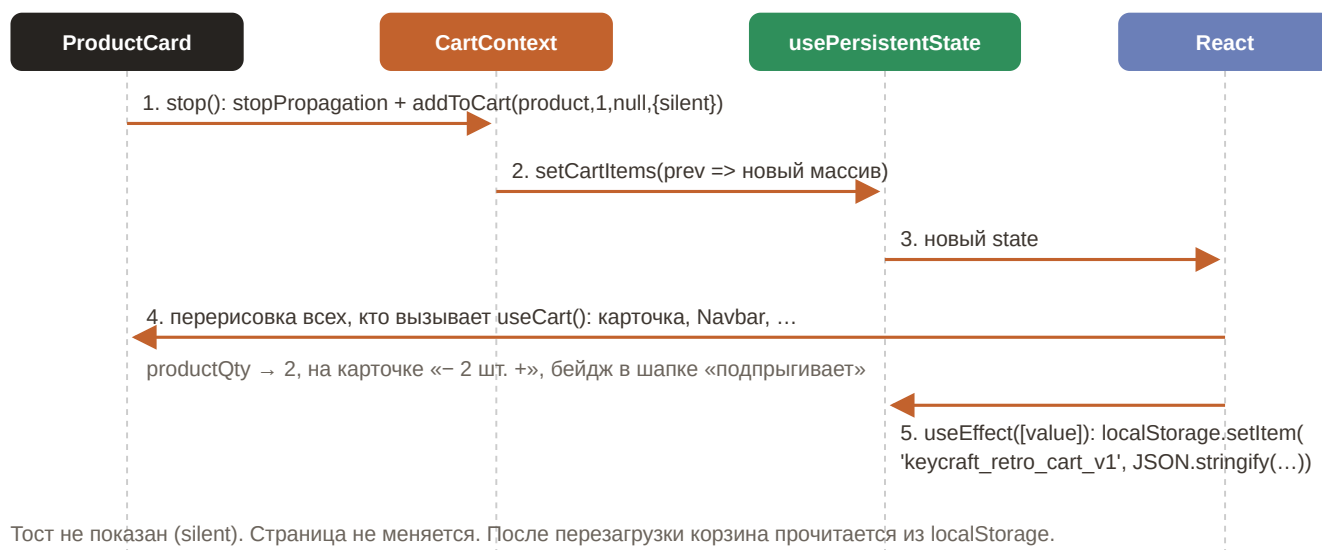


Рис. 6. Путь клика «+» через контекст, React и хранилище

13.2 «Перезагрузил страницу на /#product/sw-oil-king»

- Браузер грузит index.html → main.jsx → App. Все state получают начальные значения: `currentPage = 'home'`.
- `CartProvider` и `FavoritesProvider` при первом рендере читают localStorage (ленивая инициализация) — корзина и избранное на месте.
- После первой отрисовки срабатывает эффект в MainShop: `syncFromHash()` видит `product/sw-oil-king` → `go('product', 'sw-oil-king')` → state меняется → открывается страница товара.
- Чат-бот, открытые окна, введенные в форму данные — исчезли (это обычный state).

13.3 «Каталог, стр. 3 → товар → «Вернуться в каталог»»

- Клик по карточке → `openProduct(id)` → `go()` записывает `scrollMemory['catalog'] = 1200`, ставит `pendingScroll = 0`.
- Каталог удаляется из DOM, его настройки остаются в sessionStorage. Страница товара открывается в начале (`useLayoutEffect` → `scrollTo(0)`).
- «Вернуться в каталог» → `backToCatalog()` → `go(..., {restore: true})` → `pendingScroll = 1200`.
- Каталог создаётся заново, читает `view` из sessionStorage — страница 3, та же сортировка. `filterKey` не изменился → сброса нет.
- `useLayoutEffect` прокручивает к 1200 до показа кадра — пользователь сразу видит то же место.

13.4 «Применил промокод thock20»

- Форма в CartDrawer → `handleApplyPromo` → `applyPromo('thock20')`.
- Код приводится к `'THOCK20'`, находится в `PROMOCODES` → `setAppliedPromo({code: 'THOCK20', discountPercent: 20, ...})` (+ запись в localStorage).

3. При перерисовке пересчитываются `discountAmount` и `totalPrice`; вместо поля ввода — зелёная плашка «Промокод ТНОСК20 (-20%)».

13.5 «Оформил заказ»

1. «Перейти к оформлению» → закрыть корзину, `setIsCheckoutOpen(true)`.
2. Отправка формы → `validate()`; при ошибках — красные подписи, дальше не идём.
3. `createOrder()`: номер `KC-` + 6 случайных цифр, дата по-русски, копия позиций, суммы → дописывается в начало массива в `keycraft_retro_orders_v1` → `clearCart()`.
4. Окно показывает чек из `completedOrder` и запускает конфетти. «Печать чека» вызывает `window.print()`.

14 Стили: Tailwind CSS

Вместо отдельных CSS-файлов для каждого компонента стили пишутся классами прямо в JSX. Каждый класс — одно правило: `px-4` = отступы слева/справа 1rem, `rounded-lg` = скругление, `bg-white` = белый фон.

Приём	Пример и смысл
Адаптивность	<code>hidden md:flex</code> — скрыто на мобилке, видно с 768px. Префиксы: <code>sm</code> ≥640, <code>md</code> ≥768, <code>lg</code> ≥1024, <code>xl</code> ≥1280px.
Состояния	<code>hover:bg-stone-100</code> , <code>active:scale-[0.98]</code> , <code>disabled:opacity-40</code> , <code>focus:ring-1</code>
Группа	<code>group</code> на карточке + <code>group-hover:text-vintage-accent</code> на заголовке — заголовок меняет цвет при наведении на всю карточку
Произвольные значения	<code>min-w-[36px]</code> , <code>text-[11px]</code> , <code>bottom-[72px]</code>
Условные классы	шаблонная строка: <code>{`... \${isActive ? 'bg-stone-900 text-white' : 'bg-white'}`}</code>

14.1 tailwind.config.js — тема проекта

tailwind.config.js · строки 7–53

```
theme: {
  extend: {
    colors: {
      cream: {
        50: '#FAF8F5',
        100: '#F5F2EB',
        200: '#EBE5D8',
        300: '#DFD7C5',
      },
    },
    vintage: {
      surface: '#F7F4EE',
      card: 'FFFFFF',
      border: '#E2DBD0',
      dark: '#262320',
      muted: '#6B645C',
      accent: '#C2622D', // Тёплый янтарный
      accentHover: '#A85122',
      accentLight: '#FBF0E9',
    },
  },
},
fontFamily: {
  sans: ['Plus Jakarta Sans', 'Inter', 'system-ui', 'sans-serif'],
  mono: ['JetBrains Mono', 'ui-monospace', 'monospace'],
  retro: ['Space Grotesk', 'Plus Jakarta Sans', 'sans-serif'],
},
boxShadow: {
  'clean': '0 2px 8px rgba(0, 0, 0, 0.06)',
  'clean-hover': '0 8px 20px rgba(0, 0, 0, 0.08)',
  'card': '0 1px 3px rgba(0, 0, 0, 0.05), 0 1px 2px rgba(0, 0, 0, 0.04)',
},
keyframes: {
  'toast-in': {
    from: { opacity: '0', transform: 'translateY(12px)' },
    to: { opacity: '1', transform: 'translateY(0)' },
  },
}
```

```

    pop: {
      '0%, 100%': { transform: 'scale(1)' },
      '40%': { transform: 'scale(1.35)' },
    },
  },
  animation: {
    'toast-in': 'toast-in 0.2s ease-out',
    pop: 'pop 0.3s ease-out',
  },
},
},
},
plugins: [],

```

Собственные цвета (`bg-cream-100` , `text-vintage-accent`), три шрифта (`font-sans` , `font-mono` , `font-retro`), тени и две анимации: `animate-toast-in` (появление уведомления) и `animate-pop` (подпрыгивание счётчика).

14.2 index.css — базовые стили и свои классы

`src/index.css` · строки 42–61

```

@layer components {
  .btn-retro {
    @apply inline-flex items-center justify-center font-medium px-4 py-2 rounded-lg border border-stone-300 bg-white text-stone-800 shadow-sm transition-all duration-150 hover:bg-stone-50 hover:scale-[0.98] select-none text-sm;
  }

  .btn-retro-primary {
    @apply inline-flex items-center justify-center font-semibold px-4 py-2 rounded-lg bg-vintage-accent text-white shadow-sm transition-all duration-150 hover:bg-vintage-accentHover active:scale-[0.98] select-none text-sm;
  }

  .btn-retro-secondary {
    @apply inline-flex items-center justify-center font-medium px-3.5 py-2 rounded-lg border border-stone-300 bg-cream-100 text-stone-800 transition-all duration-150 hover:bg-cream-200 active:scale-[0.98] select-none text-sm;
  }

  .card-retro {
    @apply bg-white border border-stone-200/90 rounded-xl shadow-sm transition-all duration-200;
  }

  .card-retro-hover {
    @apply hover:shadow-md hover:-translate-y-0.5 hover:border-stone-300;
  }
}

```

`@apply` собирает несколько утилит Tailwind в один класс — так получились кнопки `btn-retro` , `btn-retro-primary` и карточка `card-retro` . В `@layer base` задаются фон страницы, фирменная полоса прокрутки на десктопе и её скрытие на мобилке (`@media (max-width: 767px)`).

15 Инструменты

15.1 Команды npm

Команда	Что делает
<code>npm install</code>	Скачать зависимости из package.json в папку <code>node_modules</code> (точные версии — в package-lock.json)
<code>npm run dev</code>	Дев-сервер Vite (обычно <code>http://localhost:5173</code>) с мгновенным обновлением при правках
<code>npm run build</code>	Собрать сайт для публикации в <code>dist/</code>
<code>npm run preview</code>	Локально открыть собранную версию из <code>dist/</code>
<code>npm run lint</code>	Проверить код линтером oxlint

15.2 Конфигурация

`vite.config.js`

```
import react from '@vitejs/plugin-react'
import { defineConfig } from 'vite'

// https://vite.dev/config/
export default defineConfig({
  plugins: [react()],
})
```

`.oxlintrc.json`

```
{
  "$schema":
    "./node_modules/oxlint/configuration_schema.json",
  "plugins": ["react", "oxc"],
  "rules": {
    "react/rules-of-hooks": "error",
    "react/only-export-components": ["warn", {
      "allowConstantExport": true
    }]
  }
}
```

Линтер с плагином `react` проверяет **правила хуков** как ошибку. Предупреждение `only-export-components` напоминает, что для Fast Refresh файл лучше экспортировать только компоненты (в контекстах рядом с провайдером экспортируются хуки `useCart` / `useFavorites` — это допустимый компромисс).

16 Слабые места и что можно улучшить

Честный список того, что стоит знать, если проект будет развиваться в настоящий магазин.

Проблема	Почему это важно	Как улучшить
Нет сервера	Заказы остаются в браузере покупателя — магазин их не получает	API (Node/Express, Supabase, Firebase) для товаров и заказов
Корзина хранит копию товара	Если поменять цену в products.js, у тех, кто уже положил товар, останется старая цена	Хранить в корзине только <code>productId</code> + вариант + количество, товар брать из каталога
Промокоды и скидки в браузере	Все коды видны в исходнике, итог можно подделать	Проверка кода и пересчёт суммы на сервере
Заказы в localStorage никто не читает	Мёртвые данные копятся	Страница «Мои заказы» или отправка на сервер
<code>count</code> в CATEGORIES вручную	Числа разойдутся с реальностью при добавлении товара	Считать: <code>PRODUCTS.filter(p => p.category === id).length</code>
<code>inStock</code> не используется	«В наличии» написано у всех	Показывать статус и блокировать кнопку
Отзывы одинаковые для всех товаров	Это заглушка	Хранить отзывы в данных товара
Валидация формы примитивная	Телефон «1234567» проходит	Маска телефона, регулярные выражения
Самописный роутинг	Растёт сложность при новых страницах	React Router (или оставить — для 4 страниц он справляется)
Нет тестов	Регрессии ловятся вручную	Vitest для контекстов (расчёт итогов), Playwright для сценариев
Всё в одном JS-файле ~375 КБ	Медленнее первая загрузка на мобильном интернете	Ленивая загрузка (<code>React.lazy</code>) для героя/оформления

17 Словарик терминов

Термин	Объяснение
SPA	Single Page Application — сайт, где страницы меняет JavaScript без перезагрузки
Компонент	Функция, возвращающая разметку (JSX). Строительный блок интерфейса
JSX	Запись HTML-подобной разметки внутри JavaScript
Props	Входные данные компонента от родителя. Только для чтения
State	Внутренняя память компонента; изменение вызывает перерисовку
Рендер	Вызов функции компонента для получения новой разметки
DOM	Дерево элементов настоящей страницы в браузере
Монтирование / размонтирование	Появление компонента на странице / удаление с неё
Хук	Функция <code>use...</code> , подключающая к компоненту возможности React
Эффект	Код, который выполняется после рендера (подписки, таймеры, хранилище)
Очистка эффекта	Функция, которую возвращает эффект; отменяет то, что он сделал
Массив зависимостей	Список значений, при изменении которых эффект/ мемо перезапускается
Ref	Изменяемый «ящик» <code>{current}</code> , переживающий рендеры без перерисовки
Контекст	Способ раздать данные всем вложенным компонентам без props
Провайдер	Компонент, который кладёт значение в контекст
Prop drilling	Передача props через много уровней, которым они не нужны
Контролируемый компонент	Компонент/поле, значение которого полностью задаётся извне
Производное значение	Значение, которое вычисляется из state, а не хранится (сумма корзины)
Неизменяемость	Обновление через создание новых объектов/массивов вместо изменения старых
key	Метка элемента списка/компонента; смена key пересоздаёт компонент
localStorage / sessionStorage	Хранилища браузера «ключ → строка»: навсегда / до закрытия вкладки
JSON.stringify / parse	Превращение объекта в строку и обратно
hash	Часть адреса после <code>#</code> ; её смена не перезагружает страницу
Всплытие события	Клик по вложенному элементу доходит и до родителей; <code>stopPropagation()</code> останавливает
requestAnimationFrame	Вызов функции перед следующим кадром экрана (~60 раз/с) — для плавной анимации
StrictMode	Режим разработки, в котором React дважды вызывает код, чтобы найти ошибки
Сборка (build)	Превращение исходников в оптимизированные файлы для публикации